
Stable and Efficient Exact Shapley Values for Product Games via Gauss–Legendre Quadrature

Anonymous Authors

Abstract

We study the exact computation of Shapley values for *product games* — cooperative games in which the coalition value factorizes as a product of per-player terms. Such games arise in machine learning explainability whenever the underlying value function inherits a multiplicative structure from the model, as in kernel methods with product kernels and tree-based models evaluated along individual decision paths. Our central observation is that, for a product game, the Shapley value of each player admits an exact one-dimensional integral representation: the weighted sum over exponentially many feature coalitions collapses to the integral of a degree- $(d - 1)$ polynomial over $[0, 1]$, where d denotes the total number of features. This yields a Gauss–Legendre quadrature scheme that is *provably exact* when the number of nodes m_q satisfies $m_q \geq \lceil d/2 \rceil$, and provides a principled approximation with error decaying geometrically in m_q otherwise — an approximation regime that is practically highly effective, as we verify empirically across models with thousands of features using only a few hundred nodes. Building on this formulation, we derive a numerically stable implementation via log-space evaluation, and an efficient parallel implementation based on associative scan primitives. Under a parallel execution model, the algorithm achieves $O(d m_q)$ total work and $O(\log d)$ parallel time. Applied to tree ensembles, the exact variant matches the complexity of Linear TreeSHAP, while remaining numerically stable, and our benchmarks show it is the fastest exact method across all tested configurations.

1 Introduction

Shapley values are a foundational tool for explaining predictions of machine learning models, providing feature attributions with strong axiomatic guarantees from cooperative game theory [Shapley, 1953]. Their principled nature has led to widespread adoption across explainable AI, both in model-agnostic methods [Lundberg and Lee, 2017, Muschalik et al., 2024, Fumagalli et al., 2026] and in model-specific methods that exploit structural properties of particular model classes, including TreeSHAP for tree ensembles [Lundberg et al., 2020, Bifet et al., 2022a], GPSHAP and variants for Gaussian processes [Chau et al., 2023, Mohammadi et al., 2026], and RKHS-SHAP for kernel methods [Chau et al., 2022]. This breadth of use has made Shapley-based attribution a central paradigm for interpreting modern machine learning systems.

Despite their broad applicability, exact Shapley values are generally expensive to compute, since the definition involves exponentially many feature coalitions. To overcome this, several model-specific methods exploit structural properties of the model under explanation in order to compute exact Shapley values in polynomial time. One particularly important case arises when the underlying model induces a multiplicative structure in the coalition value function used for Shapley attribution. This connects naturally to *product games*, in which the value of a coalition factorizes across players [Dehez, 2024]. Moreover, since the Shapley value is linear in the value function, the same perspective extends directly to weighted sums of product games. Such multiplicative value functions arise in

several important explainability settings, including path-dependent tree-based models [Lundberg et al., 2020, Bifet et al., 2022a] and kernel methods with product kernels [Mohammadi et al., 2025].

A common strategy for exploiting multiplicative structure reformulates the Shapley computation in terms of polynomial representations and recovers the required coefficients via interpolation. In tree-based models, recent methods leverage Chebyshev or Fourier interpolation to compute Shapley values in linear time by evaluating the underlying polynomial at carefully chosen nodes [Bifet et al., 2022a, Merrick and Taly, 2020]. These approaches are asymptotically optimal, but their numerical stability deteriorates rapidly with the degree of the interpolation polynomial: in tree ensembles the degree is governed by the path length, and in more general multiplicative settings it scales with the number of features, rendering interpolation-based methods unreliable in high dimensions. For product-kernel methods, a more stable alternative based on shared algebraic recursions has been proposed [Mohammadi et al., 2025], but at the cost of substantially more arithmetic operations. The literature thus exposes a persistent trade-off between numerical robustness and computational efficiency, with no existing method offering both.

This paper resolves that trade-off by recasting the exponentially-weighted Shapley sum for a product game as the integral of a single polynomial over $[0, 1]$. The representation eliminates both subset enumeration and interpolation-based coefficient recovery, and admits a Gauss–Legendre quadrature scheme that is *exact* whenever the number of nodes $m_q \geq \lceil d/2 \rceil$ and geometrically convergent otherwise; empirically, a few hundred nodes deliver near-machine precision for problems with up to 10,000 features. Two structural properties of the integrand then yield an algorithm that is simultaneously stable and fast: the integrand factorizes across features, so at each quadrature node all d Shapley values share a single length- d product computed once in log-space with explicit sign tracking; and this shared product is an associative reduction, hence amenable to parallel-scan primitives [Blelloch, 1990]. The resulting estimator attains $O(dm_q)$ total work and $O(\log d)$ parallel time under a standard parallel model. Specialized to tree ensembles, it matches the asymptotic complexity of Linear TreeSHAP while remaining numerically stable, and our benchmarks show it is the fastest *numerically correct* exact method across every tested configuration, outperforming SHAP by $3\text{--}5\times$ at large ensemble sizes while maintaining additivity residuals at floating-point precision.

2 Preliminaries

Notation Let $\mathcal{D} = \{1, \dots, d\}$ denote the set of d input features and $2^{\mathcal{D}}$ its power set. The training set consists of n samples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \mathbb{R}$ (or a discrete label set in classification). We write $\mathbf{X} \in \mathbb{R}^{n \times d}$ for the feature matrix. For a subset $\mathcal{S} \subseteq \mathcal{D}$, $\mathbf{X}_{\mathcal{S}}$ denotes the restriction of $\mathbf{X}$ to the columns indexed by $\mathcal{S}$, and $\mathbf{x}_{\mathcal{S}}$ the restriction of a vector $\mathbf{x}$ to the same subset. We use bold lowercase letters for vectors, bold uppercase letters for matrices, calligraphic letters for sets, and capital letters for random variables. Element-wise multiplication and division are denoted by $\odot$ and $\oslash$, respectively, and expectation by $\mathbb{E}$. A symmetric positive (semi-)definite kernel function is denoted by k , with $k_{\mathcal{S}}$ indicating its restriction to features in $\mathcal{S}$. The corresponding kernel matrices are denoted by $\mathbf{K}$ and $\mathbf{K}_{\mathcal{S}}$, respectively. Proofs are deferred to Appendix A.

Shapley values. The Shapley value [Shapley, 1953] is a classical solution concept from cooperative game theory and a widely used method for feature attribution in explainable machine learning. It assigns an importance score to each feature by averaging its marginal contribution over all feature coalitions, with weights determined by coalition size. Among allocation rules for transferable-utility games, it is uniquely characterized by the axioms of *efficiency*, *symmetry*, the *null-player* property, and *linearity* [Shapley, 1953, Weber, 1988], and these axioms have been repeatedly invoked as motivation for Shapley-based attribution in explainable ML [Lundberg and Lee, 2017, Muschalik et al., 2024, Fumagalli et al., 2026, Lundberg et al., 2020, Chau et al., 2023, 2022, Sundararajan and Najmi, 2020].

Formally, given a value function $v : 2^{\mathcal{D}} \rightarrow \mathbb{R}$ that assigns a scalar contribution to each subset of features, the Shapley value of feature $j \in \mathcal{D}$ is defined as

$$\phi_j(v) = \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{j\}} \mu(|\mathcal{S}|) (v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})), \quad \mu(s) = \frac{s! (d - s - 1)!}{d!}. \quad (1)$$

The term $v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})$ is the marginal contribution of feature j to coalition $\mathcal{S}$.

Equation (1) is a sum over 2^{d-1} coalitions, so exact computation is exponential in d for generic v . Scalable Shapley attribution has therefore developed along two complementary directions: *model-agnostic* approximations such as KernelSHAP and its refinements [Lundberg and Lee, 2017, Muschalik et al., 2024, Fumagalli et al., 2026, Ribeiro et al., 2016], and *model-specific* exact algorithms that exploit algebraic or combinatorial structure in v [Lundberg et al., 2020, Bifet et al., 2022a, Merrick and Taly, 2020, Yang, 2021, Jiang et al., 2025, Mohammadi et al., 2025, 2026, Chau et al., 2022, 2023]. The methods developed in this paper belong to the model-specific line and target the broad class of value functions with *multiplicative* structure.

Value functions for explainability. In explainable ML, the value function v specifies how a predictive model f is evaluated when only a subset of features is treated as “present,” and different choices correspond to different semantics for missing features [Sundararajan and Najmi, 2020, Molnar, 2022]. The two standard choices are the *observational/conditional* value $v_{\mathbf{x}}^{\text{obs}}(\mathcal{S}) = \mathbb{E}[f(X) \mid X_{\mathcal{S}} = \mathbf{x}_{\mathcal{S}}]$, which imputes missing features from the conditional distribution, and the *interventional/marginal* value $v_{\mathbf{x}}^{\text{int}}(\mathcal{S}) = \mathbb{E}_{X_{\mathcal{D} \setminus \mathcal{S}}}[f(\mathbf{x}_{\mathcal{S}}, X_{\mathcal{D} \setminus \mathcal{S}})]$, which treats missing features as drawn from their marginal distribution independently of $\mathbf{x}_{\mathcal{S}}$ [Janzing et al., 2020, Chen et al., 2020, Heskes et al., 2020]. In this paper we focus on a broad class of value functions that exhibit multiplicative structure across features, independently of the specific semantics used for missing features. For the concrete use cases, we instantiate this framework using value functions that are already well established in the literature, including the ones for product kernels and tree-based models.

3 Shapley Values for Product Games via Gauss–Legendre Quadrature

3.1 Product games and the Shapley formula

We consider cooperative games whose coalition values factorize across features. Such multiplicative structures arise naturally in several settings, including product-form kernels and other product-structured value functions.

Definition 1 (Product game). A game $v : 2^{\mathcal{D}} \rightarrow \mathbb{R}$ is called a product game if

$$v(\mathcal{S}) = \prod_{j \in \mathcal{S}} u_j(x_j), \quad \mathcal{S} \subseteq \mathcal{D}, \quad (2)$$

with the convention $v(\emptyset) = 1$.

For notational simplicity, throughout this section we write $u_j := u_j(x_j)$. Substituting the product form equation (2) into the Shapley definition equation (1), the marginal contribution of feature i to a coalition $\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}$ becomes

$$v(\mathcal{S} \cup \{i\}) - v(\mathcal{S}) = \left(\prod_{j \in \mathcal{S}} u_j \right) (u_i - 1). \quad (3)$$

Therefore,

$$\phi_i(v) = (u_i - 1) \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}} \mu(|\mathcal{S}|) \prod_{j \in \mathcal{S}} u_j, \quad \mu(s) = \frac{s! (d - s - 1)!}{d!}. \quad (4)$$

Equation (4) isolates the main computational bottleneck for product games: a weighted sum over all subsets $\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}$, where the weights depend only on coalition size. The same bottleneck appears in existing exact Shapley methods for multiplicative models, but is handled differently depending on the representation. In tree-based models, Linear TreeSHAP [Bifet et al., 2022a, Merrick and Taly, 2020] exploits the low-dimensional product structure along each decision path and performs the required aggregation through polynomial arithmetic in interpolation form. In product-kernel methods [Mohammadi et al., 2024], the weighted subset sum is instead rewritten in terms of weighted elementary symmetric polynomials and computed exactly through recursive identities based on Newton’s identities.

Our approach takes a complementary route. Instead of recovering the weighted sum through interpolation or recursive coefficient computation, the following subsections show how the Shapley weights can be absorbed into a one-dimensional integral, leading directly to a quadrature-ready representation.

3.2 A quadrature-ready Shapley formula for product games

We now present a one-dimensional integral representation that can yield an analytic formula for $\phi_i(v)$ and enables stable and efficient computation via Gauss–Legendre quadrature.

Proposition 1. *Let $v(S) = \prod_{j \in S} u_j$ be a product game. Then, for each $i \in \mathcal{D}$,*

$$\phi_i(v) = (u_i - 1) \int_0^1 \prod_{j \neq i} ((1 - t) + t u_j) dt. \quad (5)$$

We now discuss how to evaluate the one-dimensional integral in equation (5) efficiently. Because $g_i(t) := \prod_{j \neq i} ((1 - t) + t u_j)$ is a polynomial in t of degree $d - 1$, the integral is a natural target for *Gauss–Legendre (GL) quadrature* [Davis and Rabinowitz, 1984]. GL with m_q nodes is exact for every polynomial of degree at most $2m_q - 1$; the nodes and weights are computed once via the Golub–Welsch algorithm [Golub and Welsch, 1969]. For functions analytic in a Bernstein ellipse E_ρ around $[0, 1]$ (the affine image of the standard Bernstein ellipse for $[-1, 1]$), GL quadrature achieves geometric convergence $O(\rho^{-2m_q})$ [Trefethen, 2008]. Since g_i is a polynomial, it is entire and therefore analytic on and inside every such ellipse, so this geometric rate applies unconditionally in the regime $m_q < \lceil d/2 \rceil$.

Lemma 1. *Let $v(S) = \prod_{j \in S} u_j$ be a product game and let $\{(\tau_q, \omega_q)\}_{q=1}^{m_q}$ be Gauss–Legendre nodes and weights on $[0, 1]$. Define, for each $i \in \mathcal{D}$,*

$$\hat{\phi}_i(v) := (u_i - 1) \sum_{q=1}^{m_q} \omega_q \prod_{j \neq i} ((1 - \tau_q) + \tau_q u_j). \quad (6)$$

1. (Exactness.) *If $m_q \geq \lceil d/2 \rceil$, then $\hat{\phi}_i(v) = \phi_i(v)$ for all $i \in \mathcal{D}$.*
2. (Approximation.) *For $m_q < \lceil d/2 \rceil$, let E_ρ be a Bernstein ellipse for $[0, 1]$ with parameter $\rho > 1$, and set $M_\rho := \max_{z \in E_\rho} |g_i(z)|$ where $g_i(z) := \prod_{j \neq i} ((1 - z) + z u_j)$. Since g_i is a polynomial, it is entire, hence analytic on and inside E_ρ for every $\rho > 1$. Therefore*

$$|\hat{\phi}_i(v) - \phi_i(v)| \leq |u_i - 1| \cdot C_\rho M_\rho \rho^{-2m_q},$$

where C_ρ depends only on ρ .

We empirically study the sensitivity of approximation quality to m_q in our experiments, and find that for product games arising from models with up to 10,000 features, a budget of a few hundred quadrature nodes is sufficient to achieve highly accurate Shapley value estimates — far below the exact threshold of $\lceil d/2 \rceil$. In addition, Lemma 1 converts the exponential cost of summing over 2^{d-1} coalitions into m_q evaluations of a single length- d product, and it does so in a way that degrades gracefully when m_q is chosen below the exactness threshold. Two algorithmic consequences follow immediately. First, evaluation across the d features shares work: at each node x_q , the Shapley values of *all* features can be recovered from a single length- d product via leave-one-out factorization, collapsing a naive $O(d^2 m_q)$ cost to $O(d m_q)$. Second, because the shared product is a reduction under an associative operator, it is amenable to log-space evaluation for numerical stability and to parallel-scan evaluation for speed, neither of which is straightforward for interpolation-based alternatives.

3.3 Stable and Efficient Shapley Attribution

Stable evaluation via shared log-space products. A direct implementation of the quadrature formula equation (6) faces two practical obstacles: (i) naively forming a length- $(d-1)$ leave-one-out product for every pair (i, q) costs $O(d^2 m_q)$ operations; and (ii) when d is large or some u_j are far from one, these products easily overflow or underflow in floating-point arithmetic, undermining the very numerical stability we seek to establish. Both issues are resolved by a single primitive: a shared log-space product per quadrature node.

For each node τ_q , define the per-feature factors and their full product

$$T_{q,j} := (1 - \tau_q) + \tau_q u_j, \quad P_q := \prod_{j=1}^d T_{q,j}, \quad (7)$$

so that every leave-one-out product reduces to a single division, $\prod_{j \neq i} T_{q,j} = P_q / T_{q,i}$. This collapses the (i, q) -loop from $O(d^2 m_q)$ to $O(d m_q)$. To execute the division stably, we accumulate the log-magnitude of P_q and track its sign separately,

$$\log |P_q| = \sum_{j=1}^d \log |T_{q,j}|, \quad \text{sgn}(P_q) = \prod_{j=1}^d \text{sgn}(T_{q,j}), \quad (8)$$

so that the Shapley estimator takes the numerically stable form

$$\hat{\phi}_i(v) = (u_i - 1) \sum_{q=1}^{m_q} \omega_q \text{sgn}(P_q) \text{sgn}(T_{q,i}) \exp(\log |P_q| - \log |T_{q,i}|). \quad (9)$$

In many models of interest (e.g., kernels based on distances or likelihood factors) the u_j , and hence the $T_{q,j}$, are nonnegative, and the sign bookkeeping is vacuous; retaining it ensures correctness when some u_j are negative. The degenerate case $T_{q,i} = 0$ is flagged by $\text{sgn}(P_q) = 0$ and handled separately, since the log-space accumulation is undefined at zero: all leave-one-out products then vanish except possibly at the unique zero index, where the remaining product is computed directly from equation (7).

Parallel implementation and complexity. An *associative scan* (also called prefix-sum or scan primitive) is a fundamental parallel building block: given a length- L sequence and an associative binary operator $\oplus$, it computes all prefix reductions $a_1, a_1 \oplus a_2, \dots, a_1 \oplus \dots \oplus a_L$ simultaneously. By pairing elements in a binary tree of depth $\log L$, this can be done in $O(L)$ total work and $O(\log L)$ parallel time, and the same tree structure trivially yields the full reduction $a_1 \oplus \dots \oplus a_L$ as a special case. Scan primitives are natively supported by modern GPU and TPU backends (e.g., `jax.lax.scan`, `torch.cumsum`).

The log-space quantities $\log |P_q|$ and $\text{sgn}(P_q)$ in equation (8) are reductions of associative operators—addition and multiplication—over the feature index j , and are therefore direct applications of this primitive. The same pattern reappears in the final weighted sum over quadrature nodes in equation (9), which is itself an associative reduction over m_q terms. Since the d Shapley values are independent given $\{(\log |P_q|, \text{sgn}(P_q))\}_{q=1}^{m_q}$, they are computed in parallel across i at no additional asymptotic cost. This analysis establishes the following result.

Proposition 2 (Total work and parallel time of the GL estimator). *Let d be the number of features and m_q the number of Gauss–Legendre nodes.*

1. (Sequential.) *The estimator $\{\hat{\phi}_i\}_{i=1}^d$ in equation (9) requires $O(d m_q)$ total work, and this bound is tight.*
2. (Parallel.) *The computation achieves parallel time $O(\log d + \log m_q)$ with $O(d m_q / (\log d + \log m_q))$ processors.*

In the exact regime $m_q = \lceil d/2 \rceil$, these bounds specialize to $O(d^2)$ total work and $O(\log d)$ parallel time.

The total work is tight because computing all d Shapley values requires forming $d m_q$ quadrature factors and combining them, each at least once. The parallel time combines the $O(\log d)$ feature-axis scan with the $O(\log m_q)$ reduction over quadrature nodes. We emphasize that the associative-scan formulation is unavailable to interpolation- or recursion-based exact alternatives, whose dependency chains are intrinsically sequential along the polynomial-degree (respectively, recursion) axis.

4 Product Games in ML Explainability

4.1 Product kernels and interventional value functions

Many kernel-based learning algorithms admit predictions that factorize across features when the kernel function decomposes as a product of univariate kernels, inducing a product-game structure

under standard explainability semantics. Such models typically admit a dual representation of the form

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}, \mathbf{x}^{(i)}), \quad (10)$$

where $\alpha_i \in \mathbb{R}$ are dual coefficients and $k(\mathbf{x}, \mathbf{x}') = \prod_{j=1}^d k_j(x_j, x'_j)$ is a product kernel with univariate factors k_j . This formulation covers a wide range of models, including support vector machines, kernel ridge regression, and Gaussian processes with separable covariance functions.

Kernel-specific value function. We adopt the kernel-specific value function developed for product-kernel methods [Mohammadi et al., 2025], which is obtained from a functional decomposition of the kernel functions and equals the product kernel restricted to the active features:

$$v_{\mathbf{x}}(\mathcal{S}) = \boldsymbol{\alpha}^\top \mathbf{K}_{\mathcal{S}}(\mathbf{X}_{\mathcal{S}}, \mathbf{x}_{\mathcal{S}}) = \sum_{i=1}^n \alpha_i \prod_{j \in \mathcal{S}} k_j(x_j, x_j^{(i)}), \quad (11)$$

with the convention $\prod_{j \in \emptyset} (\cdot) = 1$, so that $v_{\mathbf{x}}(\emptyset) = \sum_{i=1}^n \alpha_i$. Intuitively, features outside $\mathcal{S}$ are removed by replacing their per-feature kernel factor with a multiplicative unit; Mohammadi et al. [2025] derive equation (11) from the functional decomposition of f induced by the product kernel.

Product-game structure. Setting $u_j^{(i)} := k_j(x_j, x_j^{(i)})$, the value function equation (11) decomposes as a weighted sum of product games indexed by training points,

$$v_{\mathbf{x}}(\mathcal{S}) = \sum_{i=1}^n \alpha_i \prod_{j \in \mathcal{S}} u_j^{(i)}. \quad (12)$$

By linearity of the Shapley value in v , the attribution ϕ_ℓ for feature ℓ is the corresponding weighted sum of the per-training-point Shapley values $\phi_\ell(v^{(i)})$ of the product games $v^{(i)}(\mathcal{S}) := \prod_{j \in \mathcal{S}} u_j^{(i)}$, each of which is evaluated by Lemma 1. The resulting cost is $O(n d m_q)$ total work with the same $O(\log d)$ parallel time analyzed in Section 3.

4.2 Tree ensemble value functions and induced product structure

We briefly review how Shapley values for tree ensembles are derived from a model-specific value function, following the formulation underlying TreeSHAP and its recent extensions, such as Linear TreeSHAP Bifet et al. [2022a].

Tree-based value function. Consider a decision tree T with leaf set L . Each leaf $u \in L$ stores a prediction V_u , and we denote by P_u the root-to- u path. Following the *extended model* construction used in TreeSHAP Lundberg et al. [2020] and its variants, a feature subset S is treated as observed by resolving splits on features in S deterministically using the explained point x , while splits on features outside S are averaged using branch probabilities. Concretely, for an edge e and a sample x , let $p_e \in (0, 1)$ be the fraction of training samples taking that edge, let $f(e)$ be its split feature, and define

$$c_e(x) = \begin{cases} 1/p_e, & \text{if } x \text{ follows } e, \\ 0, & \text{otherwise.} \end{cases}$$

For each feature j , define the path factor at leaf u by $q_{j,u}(x) = \prod_{e \in P_u: f(e)=j} c_e(x)$, with the empty product equal to 1. Then the contribution of leaf u under subset S is

$$V_u \left(\prod_{e \in P_u} p_e \right) \left(\prod_{j \in S} q_{j,u}(x) \right),$$

hence

$$v_x(S) = \sum_{u \in L} V_u \left(\prod_{e \in P_u} p_e \right) \left(\prod_{j \in S} q_{j,u}(x) \right).$$

Thus, each leaf induces one product game, and the full tree value function is their sum. Notably, the game dimension is not the ambient number of features d , but the number D of distinct features appearing on a root-to-leaf path.

Shapley values in parallel. Applying Lemma 1 leaf-by-leaf and summing over leaves, we obtain the Shapley value of feature i . Let $\{(\tau_r, \omega_r)\}_{r=1}^{m_q}$ be Gauss–Legendre nodes and weights on $[0, 1]$, and define $a_r(q) = 1 - \tau_r + \tau_r q$. Then

$$\phi_i \approx \sum_{r=1}^{m_q} \omega_r \sum_{u \in L} G_u(r) \frac{q_{i,u} - 1}{a_r(q_{i,u})}, \quad \text{where} \quad G_u(r) = V_u \left(\prod_{e \in P_u} p_e \right) \prod_j a_r(q_{j,u}). \quad (13)$$

Only features that occur on the path to u contribute, since $q_{j,u} = 1$ otherwise. Direct evaluation therefore costs $O(m_q LD)$, which becomes $O(LD^2)$ in the exact setting $m_q = \lceil D/2 \rceil$: for each leaf and quadrature node, we multiply over the at most D active path features. The total work of this procedure matches that of TreeSHAP. Moreover, since the contributions of different leaves can be evaluated independently, the method is naturally parallelizable when sufficient computational resources are available. However, in a sequential setting, this direct implementation still incurs unnecessary repeated computations across leaves and features. We next show how to reorganize the computation to reduce the total runtime while preserving numerical stability.

Optimizing Shapley value evaluation We now reorganize the same computation so that the total work becomes linear in the number of leaves. Define

$$s_r(q) = \frac{q - 1}{a_r(q)}.$$

Along a root-to-leaf path, each time we traverse an edge e that splits on feature i , the current state of that feature changes from some value q_e^- to q_e^+ . Since these updates occur sequentially along the path we utilize telescoping sum technique from Linear TreeSHAP algorithm and write

$$s_r(q_{i,u}) = \sum_{e \in P_u: f(e)=i} (s_r(q_e^+) - s_r(q_e^-)),$$

that is, the leaf contribution telescopes over the edges on the path that split on feature i . Swapping the order of summation from leaves to edges gives

$$\phi_i \approx \sum_{r=1}^{m_q} \omega_r \sum_{e: f(e)=i} H_e(r) \Delta_e(r),$$

where

$$\Delta_e(r) = s_r(q_e^+) - s_r(q_e^-), \quad H_e(r) = \sum_{u \text{ below } e} G_u(r).$$

The resulting algorithm is a single DFS. Along the current path we maintain the feature states q_j and the running values

$$B_r = \left(\prod_{e \text{ on current path}} p_e \right) \prod_j a_r(q_j).$$

At a leaf u , we return $V_u B_r = G_u(r)$. When traversing an edge e on feature j , only q_j changes, so each B_r can be updated in $O(1)$ time by multiplying with $p_e a_r(q_j^+)/a_r(q_j^-)$. After the recursive call returns, we obtain the subtree sums $H_e(r)$, add them to the parent accumulator, and update ϕ_j using $\Delta_e(r)$. The pseudocode of the resulting algorithm is provided in 1.

Since each edge is processed once and every edge update costs $O(m_q)$, the total runtime is $O(m_q L)$. In the exact setting $m_q = \lceil D/2 \rceil$, this becomes $O(LD)$, matching Linear TreeSHAP while retaining numerical stability and providing a natural speed–accuracy trade-off when fewer quadrature nodes are used.

5 Experiments

Effect of quadrature budget m_q on kernel Shapley values. We evaluate how the number of Gauss–Legendre nodes m_q controls approximation quality for a kernel method. We fit a Kernel Ridge Regressor (KRR) with an RBF kernel to standardised synthetic regression data ($n=500$ samples) for each $d \in \{50, 100, 1000, 5000, 10000\}$, then draw 10 random values of m_q from

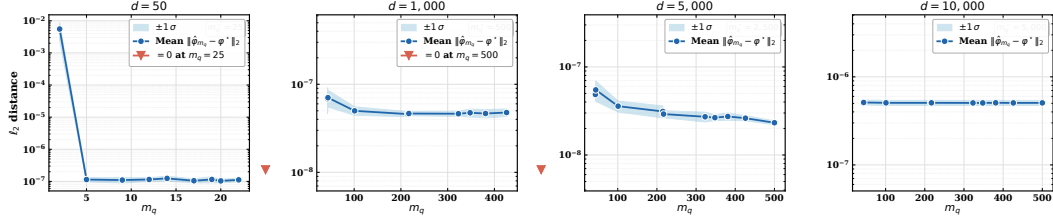


Figure 1: Convergence of the GL Shapley approximation to the exact value φ^* (computed at $m_q^* = \lceil d/2 \rceil$) for a KRR model with an RBF kernel. Each panel shows the mean ℓ_2 distance $\pm 1\sigma$ over ten test instances as m_q increases. The dashed vertical line marks m_q^* when it falls within the plotted range; otherwise its value is annotated in the top-right corner. For $d \leq 1000$ the curve reaches zero (∇) at m_q^* , confirming exactness. For $d \in \{5000, 10000\}$, only $m_q \leq 500$ is shown: the error already decreases monotonically and remains below 10^{-7} .

$[1, \min(500, \lceil d/2 \rceil)]$ and measure the mean ℓ_2 distance to the exact Shapley vector φ^* , computed once per instance at the exactness threshold $m_q^* = \lceil d/2 \rceil$. Figure 1 reports the mean distance $\pm 1\sigma$ over ten test points.

The results confirm the exponential convergence guaranteed by Lemma 1. For $d=50$, the error drops five orders of magnitude—from $\approx 5 \times 10^{-3}$ at $m_q=2$ to machine-precision level at $m_q=5$, requiring fewer than 20% of the exact nodes. For larger d , the error is already below 10^{-7} at $m_q=43$ —well under 1% of the exact threshold for $d \geq 1000$. At $d=5000$ (exact threshold $m_q^*=2500$), increasing m_q from 43 to 500 halves the error from $\approx 5 \times 10^{-8}$ to 2×10^{-8} , with monotone convergence throughout. At $d=10,000$ the sweep remains below $m_q^*=5000$, yet the absolute error stays at $\approx 5 \times 10^{-7}$ —negligible for any practical attribution task. Taken together, the results show that a budget of a few hundred quadrature nodes suffices to achieve near-exact attributions at scales far beyond the reach of exact computation.

TreeSHAP: runtime and numerical stability. We benchmark QUADRASHAP against SHAP [Lundberg and Lee, 2017], FASTTREESHAP v1/v2 [Yang, 2021], LINEAR TREESHAP [Bifet et al., 2022a,b] and SHAPIQ [Muschalik et al., 2024] on `sklearn` random forests trained on synthetic `make_regression` data. We sweep $d \in \{10, 100\}$ features and ensemble size from 10 to 10^5 leaves. For each configuration we train three independent forests, explain ten held-out samples, and report the median single-threaded wall time together with the worst-case additivity residual $\max_i |\mathbb{E}[f] + \sum_j \phi_j(x_i) - f(x_i)|$.

Table 1 summarises the results. Our method (QUADRASHAP, $m_q = \lceil D/2 \rceil$) is the fastest numerically correct method at every configuration, outperforming FASTTREESHAP v1 by roughly $2\times$ and SHAP by $3\text{--}5\times$, with the gap widening as the ensemble grows. Its additivity residual stays at floating-point precision ($\sim 10^{-13}$) throughout. LINEAR TREESHAP is competitive on wall time but suffers from catastrophic numerical breakdown: for $d=10$ its additivity residual grows from 6×10^{-14} at 10 leaves to 3×10^{-6} at 10^3 leaves and ~ 10 at 10^4 leaves, making the reported speedups meaningless at large ensemble sizes. The instability worsens with feature width: at $d=100$ and 10^5 leaves the residual reaches 7×10^2 . SHAPIQ is orders of magnitude slower and times out at 10^5 leaves for $d=10$.

6 Conclusion

We showed that Shapley values for product games reduce to a one-dimensional integral, enabling exact computation via Gauss–Legendre quadrature in $O(d m_q)$ total work and $O(\log d)$ parallel time, with geometric convergence in m_q below the exactness threshold. A log-space implementation resolves numerical instability that affects interpolation-based alternatives. Applied to tree ensembles, QUADRASHAP matches the complexity of Linear TreeSHAP while remaining numerically stable across all tested scales. Natural directions for future work include extending the quadrature framework to higher-order feature interactions [Muschalik et al., 2024], broadening the product-game formulation to additional model classes with multiplicative structure such as attention-based and multiplicative

Table 1: Single-threaded TreeSHAP runtime (ms, median over 3 sklearn RandomForestRegressors with 10 samples each) and worst-case additivity residual $\max_i |\mathbb{E}[f] + \sum_j \phi_{ij} - f(x_i)|$. Best time in each column is **bold**. **Red** residuals indicate numerical breakdown. Columns are total leaves in the ensemble.

Method	$d = 10$							$d = 100$					
	10	20	50	100	1k	10k	100k	10	20	50	100	1k	10k
TREESHAP	0.009 $5 \cdot 10^{-14}$	0.017 $6 \cdot 10^{-14}$	0.037 $1 \cdot 10^{-13}$	0.089 $1 \cdot 10^{-13}$	1.1 $3 \cdot 10^{-13}$	14.3 $1 \cdot 10^{-12}$	189 $3 \cdot 10^{-12}$	0.011 $9 \cdot 10^{-14}$	0.017 $7 \cdot 10^{-14}$	0.039 $1 \cdot 10^{-13}$	0.099 $2 \cdot 10^{-13}$	1.3 $6 \cdot 10^{-13}$	21.6 $2 \cdot 10^{-12}$
FASTTREESHAP v1	0.010 $5 \cdot 10^{-14}$	0.013 $9 \cdot 10^{-14}$	0.025 $9 \cdot 10^{-14}$	0.048 $1 \cdot 10^{-13}$	0.63 $3 \cdot 10^{-13}$	7.9 $1 \cdot 10^{-12}$	98.1 $3 \cdot 10^{-12}$	0.010 $6 \cdot 10^{-14}$	0.013 $1 \cdot 10^{-13}$	0.027 $1 \cdot 10^{-13}$	0.054 $2 \cdot 10^{-13}$	0.72 $7 \cdot 10^{-13}$	10.5 $2 \cdot 10^{-12}$
FASTTREESHAP v2	0.037 $3 \cdot 10^{-14}$	0.042 $6 \cdot 10^{-14}$	0.062 $6 \cdot 10^{-14}$	0.11 $1 \cdot 10^{-13}$	2.4 $2 \cdot 10^{-13}$	7.8 $1 \cdot 10^{-12}$	95.9 $3 \cdot 10^{-12}$	0.037 $6 \cdot 10^{-14}$	0.043 $6 \cdot 10^{-14}$	0.069 $6 \cdot 10^{-14}$	0.14 $2 \cdot 10^{-13}$	3.7 $6 \cdot 10^{-13}$	10.7 $2 \cdot 10^{-12}$
LINEAR TREESHAP	0.014 $6 \cdot 10^{-14}$	0.019 $2 \cdot 10^{-13}$	0.029 $8 \cdot 10^{-12}$	0.048 $9 \cdot 10^{-11}$	0.49 $3 \cdot 10^{-6}$	6.3 $1 \cdot 10^1$	69.0 $2 \cdot 10^1$	0.016 $6 \cdot 10^{-14}$	0.021 $2 \cdot 10^{-13}$	0.029 $1 \cdot 10^{-12}$	0.049 $1 \cdot 10^{-10}$	0.50 $2 \cdot 10^{-5}$	6.0 $6 \cdot 10^0$
SHAPIQ	3.3 $2 \cdot 10^{-13}$	7.8 $3 \cdot 10^{-13}$	24.3 $3 \cdot 10^{-13}$	53.5 $1 \cdot 10^{-12}$	576 $1 \cdot 10^{-11}$	6198 $9 \cdot 10^{-11}$	timeout	3.3 $2 \cdot 10^{-13}$	6.8 $7 \cdot 10^{-14}$	21.1 $2 \cdot 10^{-12}$	46.5 $3 \cdot 10^{-12}$	580 $3 \cdot 10^{-9}$	4597 $9 \cdot 10^{-5}$
QuadraSHAP	0.006 $4 \cdot 10^{-14}$	0.009 $6 \cdot 10^{-14}$	0.017 $6 \cdot 10^{-14}$	0.031 $1 \cdot 10^{-13}$	0.29 $2 \cdot 10^{-13}$	3.8 $6 \cdot 10^{-13}$	45.5 $8 \cdot 10^{-13}$	0.006 $6 \cdot 10^{-14}$	0.009 $6 \cdot 10^{-14}$	0.016 $2 \cdot 10^{-13}$	0.030 $1 \cdot 10^{-13}$	0.32 $2 \cdot 10^{-13}$	4.1 $4 \cdot 10^{-13}$

neural networks, and developing adaptive quadrature strategies that tune m_q per feature based on local convergence estimates.

A Proofs

Proof of Proposition 1

By definition, the Shapley value of player $i \in \mathcal{D}$ is

$$\phi_i(v) = \sum_{S \subseteq \mathcal{D} \setminus \{i\}} \frac{|S|! (d - |S| - 1)!}{d!} (v(S \cup \{i\}) - v(S)).$$

For a product game, the marginal contribution simplifies to

$$v(S \cup \{i\}) - v(S) = \left(\prod_{j \in S} u_j \right) (u_i - 1).$$

Substituting this expression yields

$$\phi_i(v) = (u_i - 1) \sum_{S \subseteq \mathcal{D} \setminus \{i\}} \mu_{|S|} \prod_{j \in S} u_j, \quad \mu_q := \frac{q! (d - q - 1)!}{d!}.$$

We now express the coefficients μ_q as integrals. For integers $q \in \{0, \dots, d-1\}$, consider the integral

$$\int_0^1 t^q (1 - t)^{d-q-1} dt.$$

By repeated integration by parts (or by the definition of the Beta function), this integral evaluates to

$$\int_0^1 t^q (1 - t)^{d-q-1} dt = \frac{q! (d - q - 1)!}{d!} = \mu_q.$$

Thus, each Shapley weight μ_q admits the representation

$$\mu_q = \int_0^1 t^q (1 - t)^{d-q-1} dt.$$

Substituting this identity into the Shapley expression and interchanging summation and integration gives

$$\phi_i(v) = (u_i - 1) \int_0^1 \sum_{S \subseteq \mathcal{D} \setminus \{i\}} t^{|S|} (1 - t)^{d-|S|-1} \prod_{j \in S} u_j dt.$$

To simplify the inner sum, observe that for each feature $j \neq i$, the term inside the summation contributes either a factor $(1 - t)$ if $j \notin S$ or a factor $t u_j$ if $j \in S$. Summing over all subsets $S \subseteq \mathcal{D} \setminus \{i\}$ therefore corresponds to expanding the product

$$\prod_{j \neq i} ((1 - t) + t u_j).$$

Since $|\mathcal{D} \setminus \{i\}| = d - 1$, this expansion yields

$$\sum_{S \subseteq \mathcal{D} \setminus \{i\}} t^{|S|} (1 - t)^{d - |S| - 1} \prod_{j \in S} u_j = \prod_{j \neq i} ((1 - t) + t u_j).$$

Substituting back into the integral completes the proof:

$$\phi_i(v) = (u_i - 1) \int_0^1 \prod_{j \neq i} ((1 - t) + t u_j) dt.$$

Proof of Lemma 1

Part (i): Exactness. The integrand $g_i(t) := \prod_{j \neq i} ((1 - t) + t u_j)$ is a polynomial of degree at most $d - 1$ in t . An m_q -point GL rule on $[0, 1]$ integrates exactly every polynomial of degree at most $2m_q - 1$ [Davis and Rabinowitz, 1984]. For $m_q \geq \lceil d/2 \rceil$ we have $2m_q - 1 \geq 2\lceil d/2 \rceil - 1 \geq d - 1$, so the quadrature sum reproduces the exact integral:

$$\sum_{q=1}^{m_q} \omega_q g_i(\tau_q) = \int_0^1 g_i(t) dt.$$

Multiplying both sides by $(u_i - 1)$ and applying Proposition 1 gives $\hat{\phi}_i(v) = \phi_i(v)$.

Part (ii): Approximation. Let

$$g_i(t) := \prod_{j \neq i} ((1 - t) + t u_j).$$

By Proposition 1, the exact Shapley value admits the integral representation

$$\phi_i(v) = (u_i - 1) \int_0^1 g_i(t) dt,$$

while its m_q -point Gauss–Legendre approximation is

$$\hat{\phi}_i(v) = (u_i - 1) \sum_{q=1}^{m_q} \omega_q g_i(\tau_q).$$

Therefore,

$$|\phi_i(v) - \hat{\phi}_i(v)| = |u_i - 1| \left| \int_0^1 g_i(t) dt - \sum_{q=1}^{m_q} \omega_q g_i(\tau_q) \right|.$$

Now g_i is a polynomial in t of degree at most $d - 1$, hence it is entire and in particular analytic on and inside every Bernstein ellipse E_ρ , for any $\rho > 1$. Standard Gauss–Legendre quadrature error bounds for analytic functions on $[-1, 1]$ [?, Thm. 19.3], transferred to $[0, 1]$ via the affine change of variables $x = 2t - 1$, imply

$$\left| \int_0^1 g_i(t) dt - \sum_{q=1}^{m_q} \omega_q g_i(\tau_q) \right| \leq \frac{4 \max_{z \in E_\rho} |g_i(z)|}{\rho^{2m_q} (\rho^2 - 1)}.$$

Substituting this bound into the previous display yields

$$|\phi_i(v) - \hat{\phi}_i(v)| \leq |u_i - 1| \frac{4 \max_{z \in E_\rho} |g_i(z)|}{\rho^{2m_q} (\rho^2 - 1)}.$$

Hence, for any fixed $\rho > 1$,

$$|\phi_i(v) - \hat{\phi}_i(v)| \leq C_\rho M_\rho |u_i - 1| \rho^{-2m_q},$$

where $M_\rho := \max_{z \in E_\rho} |g_i(z)|$ and $C_\rho := 4/(\rho^2 - 1)$ depends only on ρ . This proves the claimed geometric decay of the approximation error. $\square$

Proof of Proposition 2

Work. Computing, for each $q \in \{1, \dots, m_q\}$, the factors $T_{q,j} = (1 - \tau_q) + \tau_q u_j$ and the associated quantities $\log |T_{q,j}|$ and $\text{sgn}(T_{q,j})$ costs $O(dm_q)$ arithmetic operations in total. Accumulating $\log |P_q| = \sum_j \log |T_{q,j}|$ and $\text{sgn}(P_q) = \prod_j \text{sgn}(T_{q,j})$ is a length- d reduction for each of the m_q nodes, contributing $O(dm_q)$ additional work. Evaluating equation (9) then requires, for each $i \in \{1, \dots, d\}$, a weighted sum of m_q terms, each obtained from $\log |P_q| - \log |T_{q,i}|$ in constant time, for a further $O(dm_q)$ operations. The total is therefore $O(dm_q)$.

Parallel time. Given an associative operator $\oplus$ and a length- L sequence, a hierarchical parallel scan evaluates $a_1 \oplus \dots \oplus a_L$ in parallel time $O(\log L)$ using $O(L)$ processors [Blelloch, 1990]. Both $\log |P_q|$ (under addition) and $\text{sgn}(P_q)$ (under multiplication) are such reductions over the feature axis of length d , and the m_q nodes are independent, so the shared-product phase completes in parallel time $O(\log d)$. For each i , evaluating equation (9) requires (i) subtracting $\log |T_{q,i}|$ from $\log |P_q|$ pointwise (parallel time $O(1)$), and (ii) a weighted sum over q , which is a length- m_q tree reduction with parallel time $O(\log m_q)$. Since the d Shapley values are independent given $\{(\log |P_q|, \text{sgn}(P_q))\}_{q=1}^{m_q}$, they are evaluated in parallel across i without increasing parallel time. The overall parallel time is therefore $O(\log d + \log m_q)$. The required number of processors is the ratio of total work to parallel time [?], giving $O(dm_q/(\log d + \log m_q))$ processors. The exact regime $m_q = \lceil d/2 \rceil$ is a special case. $\square$

B Pseudocode for tree algorithms

Complexity. When DFS moves across an edge, only one q_j changes, so updating the running quantities costs $O(m_q)$. Each subtree aggregation and Shapley update also costs $O(m_q)$ per edge. Since every edge is traversed once downward and once upward, the total running time is $O(m_q L)$.

Numerical note. As before, the updates for q and G can be carried out in log-space: store $\log |G_r|$ (and, if needed, $\log |q_j|$), together with sign bits. Then multiplications/divisions become additions/subtractions of logs. Zero values should be handled explicitly rather than by taking logarithms, and special care is needed when $a_r(q) = 0$.

References

- Albert Bifet, Jesse Read, Chao Xu, et al. Linear treeshap. In *Advances in Neural Information Processing Systems*, 2022a.
- Albert Bifet, Jesse Read, Chao Xu, et al. Linear treeshap: reference implementation. https://github.com/yupbank/linear_tree_shap, 2022b. Accessed: 2026-04-12, commit: 9deb5568.
- Guy E. Blelloch. Prefix sums and their applications. Technical report, Carnegie Mellon University, 1990.
- Siu Lun Chau, Robert Hu, Javier Gonzalez, and Dino Sejdinovic. Rkhs-shap: Shapley values for kernel methods. *Advances in neural information processing systems*, 35:13050–13063, 2022.
- Siu Lun Chau, Krikamol Muandet, and Dino Sejdinovic. Explaining the uncertain: Stochastic shapley values for gaussian process models. *Advances in Neural Information Processing Systems*, 36: 50769–50795, 2023.
- Hugh Chen, Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. True to the model or true to the data? *arXiv preprint arXiv:2006.16234*, 2020.
- Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Academic Press, 2nd edition, 1984.
- Pierre Dehez. Cooperative product games. *International Game Theory Review*, 26(01):2350016, 2024.
- Fabian Fumagalli, R Teal Witter, and Christopher Musco. Polyshap: Extending kernelshap with interaction-informed polynomial regression. *arXiv preprint arXiv:2601.18608*, 2026.

- Gene H. Golub and John H. Welsch. Calculation of Gauss quadrature rules. *Mathematics of Computation*, 23(106):221–230, 1969.
- Tom Heskes, Enzo Sijben, Ioana Bucur, and Tom Claassen. Causal shapley values: Exploiting causal knowledge to explain individual predictions of complex models. *Advances in Neural Information Processing Systems*, 33, 2020.
- Dominik Janzing, Lenon Minorics, and Patrick Blöbaum. Feature relevance quantification in explainable ai: A causal problem. *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*, 2020.
- Zhongli Jiang, Min Zhang, and Dabao Zhang. Fast calculation of feature contributions in boosting trees. In *The 41st Conference on Uncertainty in Artificial Intelligence*, 2025.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 2017.
- Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable ai for trees. *Nature Machine Intelligence*, 2020.
- Luke Merrick and Ankur Taly. The explanation game: Explaining machine learning models using shapley values. *arXiv preprint arXiv:2009.05500*, 2020.
- Majid Mohammadi, Siu Lun Chau, and Krikamol Muandet. Linear-time exact shapley values for product kernel methods. *arXiv preprint*, 2024.
- Majid Mohammadi, Siu Lun Chau, and Krikamol Muandet. Computing exact shapley values in polynomial time for product-kernel methods. *arXiv preprint arXiv:2505.16516*, 2025.
- Majid Mohammadi, Krikamol Muandet, Ilaria Tiddi, Annette Ten Teije, and Siu Lun Chau. Exact shapley attributions in quadratic-time for fanova gaussian processes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 24414–24421, 2026.
- Christoph Molnar. *Interpretable Machine Learning*. Lulu.com, 2022. URL <https://christophm.github.io/interpretable-ml-book/>.
- Maximilian Muschalik, Hubert Baniecki, Fabian Fumagalli, Patrick Kolpaczki, Barbara Hammer, and Eyke Hüllermeier. shapiq: Shapley interactions for machine learning. *Advances in Neural Information Processing Systems*, 37:130324–130357, 2024.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *KDD*, 2016.
- Lloyd S. Shapley. A value for n-person games. In *Contributions to the Theory of Games*. Princeton University Press, 1953.
- Mukund Sundararajan and Amir Najmi. The many shapley values for model explanation. *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Lloyd N. Trefethen. Is Gauss quadrature better than Clenshaw–Curtis? *SIAM Review*, 50(1):67–87, 2008.
- Robert J. Weber. Probabilistic values for games. *The Shapley Value: Essays in honor of Lloyd S. Shapley*, pages 101–119, 1988.
- Jilei Yang. Fast treeshap: Accelerating shap value computation for trees. *arXiv preprint arXiv:2109.09847*, 2021.

Algorithm 1 Optimal DFS algorithm

Require: Tree T , explained sample x , quadrature nodes and weights $\{(\tau_r, \omega_r)\}_{r=1}^{m_q}$

Ensure: Shapley values ϕ_i

Definitions: $a_r(q) = 1 - \tau_r + \tau_r q$, $s_r(q) = \frac{q - 1}{a_r(q)}$

State along current DFS path:
 $q[j]$: current factor of feature j (default 1)
 $B[r]$: current product value $\left(\prod_{e \text{ on path}} p_e\right) \prod_j a_r(q[j])$

```
1: for all features  $i$  do
2:    $\phi_i \leftarrow 0$ 
3: end for
4: for all features  $j$  do
5:    $q[j] \leftarrow 1$ 
6: end for
7: for  $r = 1$  to  $m_q$  do
8:    $B[r] \leftarrow 1$ 
9: end for
10: function DFS( $u$ )
11:   if  $u$  is a leaf then
12:     for  $r = 1$  to  $m_q$  do
13:        $H[r] \leftarrow V_u \cdot B[r]$ 
14:     end for
15:     return  $H$ 
16:   end if
17:   for  $r = 1$  to  $m_q$  do
18:      $H_{\text{tot}}[r] \leftarrow 0$ 
19:   end for
20:   for all child edges  $e = (u, v)$  do
21:      $j \leftarrow f(e)$ 
22:      $p \leftarrow p_e$ 
23:      $c \leftarrow c_e(x)$ 
24:      $q_{\text{old}} \leftarrow q[j]$ 
25:      $q_{\text{new}} \leftarrow q_{\text{old}} \cdot c$ 
26:     for  $r = 1$  to  $m_q$  do
27:        $B[r] \leftarrow B[r] \cdot p \cdot \frac{a_r(q_{\text{new}})}{a_r(q_{\text{old}})}$ 
28:     end for
29:      $q[j] \leftarrow q_{\text{new}}$ 
30:      $H_{\text{child}} \leftarrow \text{DFS}(v)$ 
31:     for  $r = 1$  to  $m_q$  do
32:        $\Delta \leftarrow s_r(q_{\text{new}}) - s_r(q_{\text{old}})$ 
33:        $\phi_j \leftarrow \phi_j + \omega_r H_{\text{child}}[r] \Delta$ 
34:        $H_{\text{tot}}[r] \leftarrow H_{\text{tot}}[r] + H_{\text{child}}[r]$ 
35:     end for
36:      $q[j] \leftarrow q_{\text{old}}$ 
37:     for  $r = 1$  to  $m_q$  do
38:        $B[r] \leftarrow B[r] / \left(p \cdot \frac{a_r(q_{\text{new}})}{a_r(q_{\text{old}})}\right)$ 
39:     end for
40:   end for
41:   return  $H_{\text{tot}}$ 
42: end function
43: DFS(root( $T$ ))
```
